

Présentation de 45 minutes NER wolof avec un CRF

COMMENT UTILISER CE DOCUMENT

Chaque page correspond à un moment de l'exposé. Le bloc vert contient une formulation prête à prononcer. Le bloc bleu indique l'écran à montrer. Les temps incluent les explications, les changements d'intervenant et les petites démonstrations.

Partie	Durée	Pages
Introduction et architecture	3 min	2-3
Données et format BIO	6 min	4-6
Caractéristiques et préparation X/y	6 min	7-9
Fonctionnement et entraînement du CRF	7 min	10-13
Évaluation et analyse sur dev	7 min	14-17
Sélection, baseline et test final	6 min	18-20
Prédiction et démonstration Streamlit	6 min	21-23
Limites et conclusion	4 min	24-25
TOTAL	45 min	

RÈGLE POUR LE GROUPE

Ne lisez pas tout mot à mot. Gardez les tournures, regardez le jury et utilisez les cellules comme preuves. Si vous prenez de l'avance, développez le bloc « si le jury demande ».

1. Ouvrir la présentation

Temps de cette partie : 1 min 30

Temps cumulé conseillé : 1 min 30

À MONTRER

Notebook : cellules 1-2.

Titre, objectif et quatre catégories.

À DIRE À L'ORAL

« Bonjour à toutes et à tous. Notre projet porte sur la reconnaissance d'entités nommées en wolof. Concrètement, nous voulons qu'une machine puisse lire une phrase et repérer quatre catégories : les personnes, les lieux, les organisations et les dates. Par exemple, dans une phrase contenant Maki Sall, Dakar, ONU et 12 mars 2024, le système doit retrouver chaque élément et lui attribuer le bon type. Nous avons choisi de construire une solution simple, mesurable et défendable, basée sur un CRF. Pendant cette présentation, nous allons montrer les données, le raisonnement, le code, les résultats, les erreurs et enfin l'application. »

SI LE JURY DEMANDE

NER signifie quoi ?

Named Entity Recognition, ou reconnaissance d'entités nommées.

TRANSITION

« Je commence par vous donner une vue d'ensemble du projet, afin que chaque fichier ait ensuite une place claire. »

2. Donner la carte du projet

Temps de cette partie : 1 min 30

Temps cumulé conseillé : 3 min

À MONTRER

Arborescence du dépôt : data/raw, src, models, projet_nlp.ipynb et app.py.

À DIRE À L'ORAL

« Le projet est organisé en cinq blocs. Le dossier data contient le corpus. Le dossier src contient les fonctions Python réutilisables. Le notebook présente le raisonnement dans l'ordre et exécute les expériences. Le dossier models contient le modèle entraîné et ses résultats finaux. Enfin, app.py fournit l'interface Streamlit. Cette séparation est volontaire : le notebook explique et prouve, tandis que les fichiers Python rendent le code propre et réutilisable. Ainsi, si le jury demande une preuve, nous montrons la cellule ; s'il demande l'implémentation, nous ouvrons le fichier associé. »

PREUVE DANS LE PROJET

Notebook : 39 cellules progressives.
src : 11 modules spécialisés.

TRANSITION

« Maintenant que l'architecture est claire, regardons la matière première du modèle : les données. »

3. Expliquer les trois ensembles de données

Temps de cette partie : 2 min

Temps cumulé conseillé : 5 min

À MONTRER

Notebook : cellules 5-6.

Fichiers : data/raw/train.txt, dev.txt et test.txt.

À DIRE À L'ORAL

« Nous séparons strictement les données en trois ensembles. Train sert à apprendre les paramètres du CRF. Dev sert à comparer les configurations et à prendre des décisions. Test reste fermé pendant toute cette phase ; il est utilisé une seule fois après avoir figé la configuration. Cette séparation évite de choisir un modèle simplement parce qu'il réussit sur les données finales. C'est un principe essentiel : train apprend, dev conseille et test juge. Dans le notebook, nous chargeons d'abord seulement train et dev. Le résultat test est ensuite relu depuis les métadonnées finales. »

SI LE JURY DEMANDE

Pourquoi ne pas entraîner sur les trois fichiers ?

Parce que nous n'aurions plus d'ensemble indépendant pour mesurer la généralisation.

TRANSITION

« Avant d'apprendre, il faut aussi comprendre le format exact d'une phrase dans le corpus. »

4. Lire le format CoNLL

Temps de cette partie : 2 min

Temps cumulé conseillé : 7 min

À MONTRER

Notebook : cellules 5-8.

Fichier : src/analyze_data.py - fonction load_conll.

Ligne CoNLL	Lecture
Maki B-PER	token Maki, début d'une personne
Sàll I-PER	suite de la même personne
dem O	mot hors entité
Dakar B-LOC	début d'un lieu

À DIRE À L'ORAL

« Le corpus utilise le format CoNLL. Chaque ligne contient un token et un label, et une ligne vide sépare deux phrases. Notre fonction load_conll lit le fichier ligne par ligne, vérifie qu'il existe exactement deux colonnes utiles et refuse les labels inconnus. Elle construit ensuite un objet Sentence avec deux séquences : tokens et labels. Nous avons aussi prévu le cas où le dernier bloc n'est pas suivi d'une ligne vide. Cette étape paraît simple, mais elle protège tout le pipeline : si les données sont mal lues, les caractéristiques, l'entraînement et les métriques deviennent faux. »

TRANSITION

« Les labels que nous venons de voir suivent un schéma précis appelé BIO. »

5. Défendre le schéma BIO

Temps de cette partie : 2 min

Temps cumulé conseillé : 9 min

À MONTRER

Notebook : cellules 7-8.

Fichier : analyze_data.py - find_bio_errors.

À DIRE À L'ORAL

« BIO permet de représenter les limites d'une entité. B signifie Beginning : le début. I signifie Inside : la continuation. O signifie Outside : hors entité. Cette distinction est importante pour les noms composés. Maki reçoit B-PER et Sàll reçoit I-PER ; ensemble, ils forment une seule personne. Nous vérifions également les transitions : un I-LOC, par exemple, ne peut pas continuer une personne. La fonction find_bio_errors repère ces incohérences sans modifier silencieusement le corpus. Nous préférons signaler un problème plutôt que corriger une réponse sans justification. »

SI LE JURY DEMANDE

Pourquoi ne pas utiliser seulement PER, LOC, ORG et DATE ?

Parce que nous perdrons les limites des entités composées de plusieurs tokens.

TRANSITION

« Après la structure, nous regardons la distribution réelle des labels et des entités. »

6. Montrer le déséquilibre du corpus

Temps de cette partie : 2 min

Temps cumulé conseillé : 11 min

À MONTRER

Notebook : cellules 9-11.

Comptage des labels, des entités et des longueurs de phrases.

À DIRE À L'ORAL

« L'analyse montre que le label O est majoritaire. C'est logique, car la plupart des mots d'une phrase ne sont pas des entités. Mais cette majorité crée un piège : un modèle qui prédit O très souvent pourrait obtenir une accuracy apparemment correcte. C'est pourquoi nous comptons les entités à partir des labels B et nous choisissons plus tard le F1 strict comme métrique principale. Nous regardons aussi la fréquence de chaque type. Les catégories rares sont naturellement plus difficiles, car le modèle observe moins d'exemples pour apprendre leurs formes et leurs contextes. »

SI LE JURY DEMANDE

Pourquoi l'accuracy est-elle trompeuse ?

Elle récompense fortement les nombreux O, même lorsque les entités sont mal reconnues.

TRANSITION

« Nous avons maintenant des phrases propres. Il faut les convertir en informations numériques et symboliques pour le CRF. »

7. Introduire les caractéristiques

Temps de cette partie : 2 min

Temps cumulé conseillé : 13 min

À MONTRER

Notebook : cellules 12-13.

Fichier : src/features.py.

Caractéristique	Exemple Sénégal	Intérêt
word.lower	sénégal	uniformiser la casse
prefix/suffix	sé / gal	formes fréquentes
word.shape	Xxxxxxx	structure du token
istitle / isdigit	True / False	nom ou nombre

À DIRE À L'ORAL

« Un CRF ne comprend pas directement le sens d'un mot. Nous lui donnons donc des indices simples et observables. Pour chaque token, nous utilisons le mot en minuscules, ses préfixes, ses suffixes, sa forme générale, sa casse et la présence de chiffres. Par exemple, une majuscule initiale peut être un indice de personne ou de lieu, tandis qu'un nombre peut être un indice de date. Aucun indice n'est une réponse certaine ; le modèle apprend leur importance à partir de train. Cette approche reste interprétable : chaque caractéristique peut être montrée et expliquée. »

TRANSITION

« À la forme du mot, nous ajoutons maintenant son contexte immédiat. »

8. Justifier le voisinage local

Temps de cette partie : 2 min

Temps cumulé conseillé : 15 min

À MONTRER

Fichier : features.py - token_features.

Caractéristiques -1 et +1, BOS et EOS.

À DIRE À L'ORAL

« Le même token peut changer de rôle selon son voisinage. Nous ajoutons donc le mot précédent et le mot suivant, avec quelques informations sur leur casse. Nous marquons aussi le début et la fin de phrase avec BOS et EOS. Ce contexte reste volontairement local : il enrichit la décision sans rendre le modèle opaque. Par exemple, un mot situé après un titre ou avant un verbe peut recevoir un indice différent. Le CRF combine ensuite ces indices avec les transitions de labels. Cette combinaison explique pourquoi il peut mieux généraliser qu'une simple mémorisation des mots vus dans train. »

SI LE JURY DEMANDE

Le CRF comprend-il vraiment le sens ?

Non. Il exploite surtout des régularités de forme, de position et de voisinage local.

TRANSITION

« Ces caractéristiques constituent X. Il reste à construire y et à garantir leur alignement. »

9. Construire X et y sans désalignement

Temps de cette partie : 2 min

Temps cumulé conseillé : 17 min

À MONTRER

Notebook : cellule 13.

Fichier : src/dataset.py - prepare_split.

À DIRE À L'ORAL

« Pour chaque phrase, X contient une liste de dictionnaires de caractéristiques, et y contient la liste des labels BIO attendus. La règle centrale est l'alignement : la position zéro de X correspond exactement à la position zéro de y, et ainsi de suite. prepare_split appelle sentence_features sur les tokens et copie les labels dans une liste. Nous vérifions ensuite le nombre de phrases et le nombre de tokens. Cette vérification évite une erreur grave : apprendre un label avec les caractéristiques d'un autre mot. Les vrais labels n'entrent jamais dans X, donc il n'y a pas de fuite de réponse. »

PREUVE DANS LE PROJET

Invariants : $\text{len}(X) == \text{len}(y)$ et, pour chaque phrase i , $\text{len}(X[i]) == \text{len}(y[i])$.

TRANSITION

« X et y sont prêts. Nous pouvons enfin présenter le cœur du projet : le CRF. »

10. Donner l'intuition du CRF

Temps de cette partie : 2 min

Temps cumulé conseillé : 19 min

À MONTRER

Notebook : cellules 14-17.

Fichier : src/train_crf.py.

À DIRE À L'ORAL

« Le CRF est un modèle de séquence. Contrairement à un classifieur qui traiterait chaque mot isolément, il cherche la meilleure séquence complète de labels pour toute la phrase. Il associe un score à chaque choix possible et retient la séquence globale la plus cohérente. Cela convient bien au BIO, car le label d'un mot dépend aussi du label précédent. Si un mot commence une personne avec B-PER, le mot suivant peut naturellement continuer avec I-PER. Le CRF apprend cette cohérence directement dans les données. »

SI LE JURY DEMANDE

Pourquoi ce modèle plutôt qu'une règle fixe ?

Parce qu'il apprend des poids à partir des exemples et combine plusieurs indices parfois contradictoires.

TRANSITION

« Pour comprendre son score, séparons maintenant émissions et transitions. »

11. Expliquer émissions et transitions

Temps de cette partie : 2 min

Temps cumulé conseillé : 21 min

À MONTRER

Notebook : cellules 16-17.

Affichage des poids de transition appris.

Composant	Question simple	Exemple
Émission	ce mot ressemble-t-il à ce label ?	chiffre → DATE
Transition	ce label suit-il bien le précédent ?	B-PER → I-PER

À DIRE À L'ORAL

« Le score d'une séquence vient de deux familles de poids. Les émissions mesurent la compatibilité entre les caractéristiques d'un token et un label. Une majuscule, un suffixe ou un chiffre peuvent favoriser certains labels. Les transitions mesurent la compatibilité entre deux labels voisins. Une transition B-PER vers I-PER peut recevoir un poids positif, tandis qu'une transition incohérente sera moins favorisée. Dans le notebook, nous affichons les transitions les plus fortes pour prouver que le modèle a réellement appris une structure de séquence, et pas seulement une liste de mots. »

TRANSITION

« Ces poids sont appris par fit, mais leur complexité doit être contrôlée par la régularisation. »

12. Défendre les hyperparamètres

Temps de cette partie : 2 min

Temps cumulé conseillé : 23 min

À MONTRER

Fichier : train_crf.py - build_crf.

L-BFGS, c1, c2, 100 itérations et toutes transitions.

À DIRE À L'ORAL

« Nous utilisons l'algorithme L-BFGS pour optimiser les poids. c1 correspond à la régularisation L1 : elle peut ramener certains poids à zéro. c2 correspond à la régularisation L2 : elle réduit les poids trop grands. Ces deux valeurs limitent le surapprentissage, c'est-à-dire la mémorisation excessive de train. Nous fixons max_iterations à 100 pour borner l'optimisation. Enfin, all_possible_transitions permet au modèle de considérer toutes les transitions de labels, y compris celles qui sont rares. Les valeurs finales ne sont pas choisies sur test, mais comparées sur dev. »

SI LE JURY DEMANDE

Pourquoi ne pas tester des centaines de configurations ?

Notre comparaison est volontairement limitée, lisible et reproductible pour ce projet.

TRANSITION

« Avec cette configuration, fit peut apprendre les poids à partir de train. »

13. Décrire précisément fit()

Temps de cette partie : 1 min

Temps cumulé conseillé : 24 min

À MONTRER

Notebook : cellule 15.

Fichier : train_crf.py - train_crf.

À DIRE À L'ORAL

« Avant fit, la fonction vérifie que X_train et y_train ne sont pas vides et qu'ils contiennent le même nombre de phrases. Elle construit ensuite un CRF vierge et appelle model.fit. fit apprend les poids des émissions et des transitions qui rendent les vraies séquences de train plus probables. Il ne modifie pas les données et il ne voit jamais dev ou test. Une fois l'entraînement terminé, le modèle peut produire une séquence de labels pour chaque nouvelle séquence de caractéristiques. »

TRANSITION

« Nous passons maintenant à l'évaluation sur dev, qui occupe une place centrale dans notre démarche. »

14. Produire et contrôler les prédictions

Temps de cette partie : 1 min 30

Temps cumulé conseillé : 25 min 30

À MONTRER

Notebook : cellules 18-19.

`predict(X_dev)` et contrôle d'alignement.

À DIRE À L'ORAL

« Nous demandons au CRF de prédire toutes les phrases de dev. Avant de calculer un score, nous contrôlons la forme des sorties. Le nombre de phrases prédites doit correspondre au nombre de phrases de référence, et chaque phrase doit contenir un label par token. La fonction `predictions_are_aligned` automatise ce contrôle.

Nous affichons aussi un premier exemple token, vrai label et label prédit. Cette étape rend le résultat concret et permet de détecter une erreur de pipeline avant qu'elle ne soit cachée dans une moyenne. »

TRANSITION

« Une fois les sorties validées, nous pouvons définir les métriques au niveau des entités. »

15. Expliquer précision, rappel et F1

Temps de cette partie : 2 min

Temps cumulé conseillé : 27 min 30

À MONTRER

Notebook : cellules 20-22.

Fichier : src/evaluate.py.

Métrique	Formule	Lecture
Précision	$TP / (TP + FP)$	fiabilité des prédictions
Rappel	$TP / (TP + FN)$	couverture des entités
F1	$2PR / (P + R)$	équilibre global

À DIRE À L'ORAL

« La précision répond à la question : parmi les entités prédites, combien sont correctes ? Elle baisse lorsque le modèle invente des entités. Le rappel répond à la question : parmi les entités attendues, combien sont retrouvées ? Il baisse lorsque le modèle manque des entités. Le F1 combine les deux avec une moyenne harmonique. Il devient faible dès que l'une des deux métriques est faible. Nous utilisons segeval, car cette bibliothèque reconstruit les entités complètes à partir des labels BIO. »

TRANSITION

« Mais toutes les correspondances ne sont pas acceptées : notre évaluation est stricte. »

16. Justifier le mode strict IOB2

Temps de cette partie : 2 min

Temps cumulé conseillé : 29 min 30

À MONTRER

`evaluate.py : mode='strict', scheme=IOB2.`

À DIRE À L'ORAL

« Une entité est correcte seulement si trois éléments correspondent : son type, son début et sa fin. Si la vérité est Maki Sàll de type PER, mais que le modèle prédit seulement Sàll de type PER, la limite est mauvaise et l'entité est considérée fausse. Si les limites sont exactes mais que le type est LOC, elle est également fausse. Ce choix évite un score artificiellement généreux. `zero_division` égal à zéro empêche aussi une erreur lorsque le modèle ne prédit aucune entité d'un type donné. »

SI LE JURY DEMANDE

Pourquoi évaluer les entités plutôt que chaque token ?

Parce que l'objectif final est de retrouver des segments complets avec le bon type.

TRANSITION

« Le score global est utile, mais nous devons aussi comprendre les vraies erreurs du modèle. »

17. Analyser les erreurs réelles

Temps de cette partie : 2 min 30

Temps cumulé conseillé : 32 min

À MONTRER

Notebook : cellules 23-25.

Fichier : src/error_analysis.py.

Erreur	Diagnostic
invented	prédiction sans entité vraie
missed	entité vraie non retrouvée
wrong_type	mêmes limites, type différent
wrong_boundaries	limites différentes avec chevauchement

À DIRE À L'ORAL

« Nous reconstruisons d'abord les entités à partir des labels BIO. Ensuite, nous comparons les positions de début, de fin et le type. Les erreurs sont classées en quatre catégories. Une entité inventée pénalise la précision. Une entité manquée pénalise le rappel. Un mauvais type signifie que les limites sont bonnes mais la catégorie est fausse. De mauvaises limites signifient que la prédiction chevauche la vraie entité sans couvrir exactement le même segment. Cette analyse nous aide à parler des causes possibles avec des exemples, plutôt que de commenter seulement un chiffre. »

TRANSITION

« Après avoir compris dev, nous pouvons choisir la configuration finale. »

18. Sélectionner c1 et c2 sur dev

Temps de cette partie : 2 min

Temps cumulé conseillé : 34 min

À MONTRER

Notebook : cellules 26-28.

Fichier : src/select_crf.py.

À DIRE À L'ORAL

« Nous comparons quatre configurations. Pour chacune, nous entraînons réellement un nouveau CRF sur train, nous prédisons dev et nous calculons le même F1 strict. La configuration équilibrée, avec c1 égal à 0,05 et c2 égal à 0,20, obtient le meilleur F1 dev : 0,7583. Elle est donc choisie puis figée. Le mot figée est important : nous ne changeons plus les hyperparamètres après avoir observé test. Cette procédure sépare clairement le réglage et l'évaluation finale. »

PREUVE DANS LE PROJET

Fonction : select_best_crf retourne le nom, le meilleur modèle et le tableau complet des résultats.

TRANSITION

« Pour savoir si ce CRF apporte une vraie valeur, nous le comparons à une baseline simple. »

19. Comparer à la baseline

Temps de cette partie : 1 min 30

Temps cumulé conseillé : 35 min 30

À MONTRER

Notebook : cellules 34-36.

Fichier : src/baseline.py.

À DIRE À L'ORAL

« La baseline mémorise le label le plus fréquent de chaque mot dans train. Si un mot n'a jamais été vu, elle répond O. Cette méthode est simple mais constitue un point de comparaison utile. Sur dev, elle obtient un F1 de 0,6811, contre 0,7583 pour le CRF sélectionné. La baseline a une bonne précision sur les mots connus, mais son rappel est plus faible. Le CRF retrouve davantage d'entités grâce aux caractéristiques de forme, au contexte local et aux transitions BIO. »

SI LE JURY DEMANDE

Pourquoi garder une baseline faible ?

Parce qu'un score isolé ne prouve pas l'apport du modèle ; la baseline donne un minimum à dépasser.

TRANSITION

« Nous pouvons maintenant ouvrir test une seule fois et annoncer le résultat final. »

20. Présenter honnêtement le test final

Temps de cette partie : 2 min 30

Temps cumulé conseillé : 38 min

À MONTRER

Notebook : cellules 29-31.

Fichier : finalize_crf.py.

Type	Précision	Rappel	F1
LOC	0,8859	0,7725	0,8253
PER	0,7055	0,5852	0,6398
ORG	0,6129	0,3455	0,4419
DATE	0,5238	0,1571	0,2418

À DIRE À L'ORAL

« Avec la configuration figée, nous entraînons le CRF sur train, nous prédisons test et nous calculons les métriques strictes. Le résultat micro final est une précision de 0,7749, un rappel de 0,5781 et un F1 de 0,6622. Le F1 est plus bas que sur dev. Nous ne cachons pas cette baisse : elle indique que test contient des cas moins bien couverts par les régularités apprises. LOC généralise bien avec un F1 d'environ 0,8253. PER atteint environ 0,6398. ORG et DATE ont surtout un rappel faible, ce qui signifie que beaucoup d'entités attendues sont manquées. »

TRANSITION

« Le modèle final est ensuite sauvegardé pour être réutilisé sans réentraînement. »

21. Expliquer la sauvegarde et la prédiction

Temps de cette partie : 2 min

Temps cumulé conseillé : 40 min

À MONTRER

Notebook : cellules 32-33.

Fichiers : `finalize_crf.py` et `predict.py`.

À DIRE À L'ORAL

« Le fichier `crf_wolof.joblib` contient l'objet CRF entraîné. Le fichier de métadonnées JSON conserve la configuration et les métriques finales. Pour une nouvelle phrase, `predict.py` suit exactement la chaîne utile : tokeniser le texte, construire les caractéristiques, appeler `predict_single`, puis reconstruire les entités BIO. Cette séparation est pratique : nous pouvons utiliser le modèle sans relancer l'entraînement. Elle améliore aussi la reproductibilité, car les résultats et les paramètres restent associés au modèle sauvegardé. »

PREUVE DANS LE PROJET

Chaîne : texte → tokens → caractéristiques → labels BIO → entités.

TRANSITION

« Nous allons maintenant exécuter cette chaîne dans l'application Streamlit. »

22. Faire la démonstration du CRF brut

Temps de cette partie : 2 min

Temps cumulé conseillé : 42 min

À MONTRER

Application : <https://5aaxfh5jf9kkjgjna7fpso.streamlit.app/>
Désactiver d'abord le mode assisté.

À DIRE À L'ORAL

« Nous commençons par le CRF brut, car c'est ce modèle qui a été évalué dans le notebook. Nous saisissons une phrase, puis nous lançons la détection. L'application affiche le texte annoté et un tableau avec un label BIO pour chaque token. Nous pouvons alors relier la démonstration aux étapes précédentes : la même tokenisation, les mêmes caractéristiques et le même modèle sauvegardé sont utilisés. Si une entité rare est manquée, nous le signalons comme une limite normale du modèle, et non comme un problème d'interface. »

SI LE JURY DEMANDE

Phrase conseillée :
Maki Sàll ak ONU dem na Dakar ci 12 mars 2024.

TRANSITION

« *L'application propose aussi un mode assisté, mais il faut bien le distinguer du résultat scientifique.* »

23. Présenter le mode assisté sans confusion

Temps de cette partie : 2 min

Temps cumulé conseillé : 44 min

À MONTRER

Activer le mode assisté.

Fichier : src/postprocess.py.

À DIRE À L'ORAL

« Le mode assisté complète seulement certains labels O avec des règles explicables. Il reconnaît quelques villes sénégalaises connues comme lieux, les sigles en majuscules comme organisations, les nombres et noms de mois comme dates, et deux mots avec une majuscule initiale comme nom complet potentiel. Ce mode améliore la démonstration sur certains cas nouveaux, mais il n'entre pas dans le score du CRF. Le résultat scientifique reste celui du modèle brut. Cette distinction est importante pour rester honnête : les règles sont un bonus d'usage, pas une preuve supplémentaire de performance du CRF. »

SI LE JURY DEMANDE

Pourquoi ne pas évaluer le mode assisté ?

Ce serait une expérience différente, à mesurer séparément sur dev avant toute annonce sur test.

TRANSITION

« Pour conclure, nous revenons sur les limites, les apprentissages et la suite possible. »

24. Conclure en une minute

Temps de cette partie : 1 min

Temps cumulé conseillé : 45 min

À MONTRER

Notebook : cellules 37-39.

Architecture, difficultés, portée et conclusion.

À DIRE À L'ORAL

« Pour conclure, notre projet propose une chaîne complète et défendable pour le NER wolof : contrôle du corpus, caractéristiques simples, CRF séquentiel, sélection sur dev, test final strict, analyse d'erreurs, sauvegarde et application. Le CRF dépasse la baseline sur dev et reste interprétable, mais son contexte local limite la généralisation, surtout pour DATE et ORG. Une suite logique serait de comparer cette base à un transformeur multilingue ou africain préentraîné, en gardant exactement le même protocole et le même F1 strict. Notre contribution n'est donc pas seulement un score : c'est un raisonnement reproductible, une évaluation honnête et un outil testable. »

PREUVE DANS LE PROJET

Phrase finale : « Merci pour votre attention. Nous sommes prêts à répondre à vos questions et à ouvrir toute cellule ou tout fichier nécessaire. »